

SparseDR: Differentiable Rendering of Sparse Signed Distance Fields

First Author^{1,2}, Second Author², Third Author^{1,2} and

¹First Affiliation

²Second Affiliation

third@other.example.com, fourth@example.com

Abstract

We present SparseDR, a differentiable rendering implementation designed for sparse representations based on Signed Distance Fields (SDF). We leverage the Sparse Brick Set (SBS) representation and propose an adaptation of redistancing and regularization of the SDF defined on SBS. This enables our method to surpass existing differentiable rendering implementations in accuracy at limited memory budgets by increasing the effective resolution of the SDF representation.

1 Introduction

Signed distance functions (SDFs) provide a continuous, implicit representation of geometry, defining the shortest Euclidean distance from any point in space to the nearest surface, with the sign indicating interior or exterior regions. This enables gradient-based optimization by supporting differentiable rendering, where ray marching samples distances to compute density and color, facilitating inverse rendering tasks such as shape and material recovery from images.

SDF-based differentiable rendering produces high-fidelity reconstructions with smooth surfaces and topological consistency, outperforming many existing methods in handling complex topologies. Unfortunately, existing SDF-based differentiable rendering implementations are constrained in reconstruction accuracy due to high memory consumption and substantial computational demands. Our work proposes a solution to this problem, alleviating limitations through better data structures and sampling techniques:

- We propose an adaptation of SDF-based differentiable rendering to a sparse data structure, which increases the effective resolution of the SDF representation and thereby improves the surface reconstruction accuracy.
- We provide two novel algorithms of redistancing for sparse SDF: a global version, which is used once per epoch, and local version used on every iteration.

2 Related Work

[Vicini *et al.*, 2022] introduces differentiable rendering operating directly on an SDF representation. The authors mod-

fied sphere tracing so that, in addition to computing the surface intersection, it evaluates a special reparameterization of rays, enabling correct gradients with respect to shape parameters, including the visibility contribution of individual surface regions. This work also first proposed the use of a *redistancing* — a process of recomputing values on a grid such that all grid cells store correct distances to the surface. In differentiable rendering this recomputation is required because, after an optimization step and updating the distances at grid nodes, the grid no longer strictly represents an SDF. [Wang *et al.*, 2024] exploits properties of distance functions and estimate the boundary integral by an integral over a relaxed boundary, i.e., a thin band around the object silhouette. [Zhang *et al.*, 2025] propose for computing gradients in the presence of multiple surfaces along a ray path: instead of locally differentiating a single surface, the authors consider a volumetric, non-local perturbation of the surface.

All of the above methods represent the distance function volumetrically on a regular grid, which leads to three fundamental issues: (1) significant memory consumption, especially when using automatic differentiation frameworks such as DrJit [Jakob *et al.*, 2022] or PyTorch [Jason and Yang, 2024]; (2) slow ray-surface intersection based on sphere tracing; (3) an expensive redistancing algorithm that propagates modified SDF values across the entire grid.

3 Implementation Details

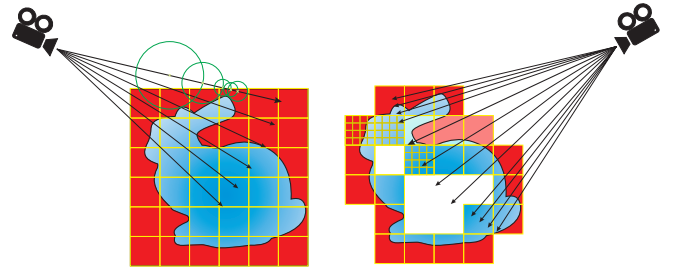


Figure 1: Comparison of SparseDR algorithm with the baseline. The baseline (on the left) uses dense SDF grid to represent the surface, and sphere tracing for ray-scene intersection. SparseDR (on the right) uses Sparse Brick Set, each brick stores small SDF grid, and a ray intersects each brick on its way until it hits the surface.

To store the surface, SparseDR uses the Sparse Brick Set (SBS) proposed in [Söderlund *et al.*, 2022], which is a hybrid of hierarchical and regular representations, see Figure 1. The entire scene is encoded as a BVH or an octree, while its leaves store small regular grids, for example 4x4x4 voxels (5x5x5 distances). For gradient computation, we adapt the relaxed boundary method from [Wang *et al.*, 2024], which we refer to as our baseline. At the beginning of the optimization process, the distance function is initialized with a sphere defined on a low-resolution grid (32x32x32). At each iteration of an epoch, we estimate the gradient and update the distance values using a gradient descent step, followed by a regularization and local redistancing (see Figure 2). On the end of each epoch two steps are executed: upsampling step and global redistancing. Upsampling is not performed before the resulting model is saved.

Gradient evaluation. The key distinction of the method underlying SparseDR lies in the transition from an SDF grid to a sparse brick set. In our case, each brick has a fixed size of 4x4x4 voxels. For the SBS, a BVH tree is constructed, with each leaf node storing a single brick. In addition to sphere tracing, the Newton method [Söderlund *et al.*, 2022] for ray-SDF intersection has been implemented. It is specifically designed to find intersections inside the voxel. A ray first traverses the BVH; if an intersection with a leaf node is found, it then intersects the voxels of the brick. The ray traverses each brick along its path until it hits the surface or there are no bricks left.

The Newton method was modified to search for relaxed boundary points. According to [Wang *et al.*, 2024], a point belongs to the relaxed boundary if the SDF value at that point is below a certain threshold, which we refer to as the relaxed epsilon, and the SDF derivative along the ray reaches its local minimum. The Newton method is based on the fact that inside a voxel, the SDF along the ray can be represented as a cubic polynomial. Thus, within the voxel, a potential relaxed boundary point is found as the root of a quadratic equation, after which the SDF value at the point is checked to be less than the relaxed epsilon. The case when the local minimum occurs on the boundary between voxels or bricks is handled separately.

SparseDR is implemented in C++ and Vulkan and does not use automatic differentiation, all derivatives were obtained manually. Gradients are accumulated in a float array whose size equals the number of optimized parameters. In the shader, when computing the color of a pixel for 16 samples, information about the voxels hit by the samples is stored; after every 16 samples (or after the last one), the gradients are added to the array. SparseDR optionally uses an additional shader that samples only pixels containing boundaries and evaluates only the boundary integral, without the need for color computation. This adaptive boundary sampling allows cheaper samples and yields more accurate gradients for smaller relaxed epsilon values.

Redistancing. For differentiable rendering of SBS in SparseDR, two versions of redistancing have been implemented. The first, full version, is an adaptation of the parallel fast sweeping method [Detrixhe *et al.*, 2013] for SBS, which is used to acquire the correct SDF. It addresses the problem

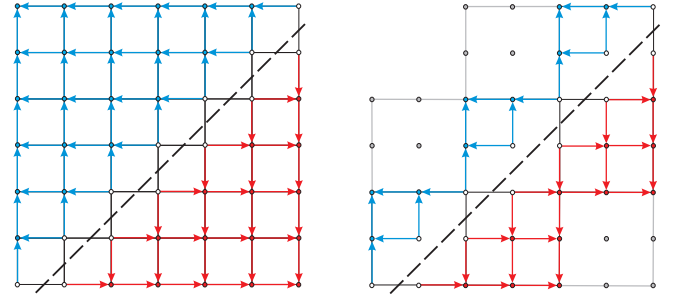


Figure 2: Comparison of the redistancing algorithm on dense grid (on the left) with the local redistancing (on the right) on SBS, bricks are 3x3 distances in size. The dots represent the distance values. Red and blue dots are the distances outside and inside the object respectively. Dashed line represents the surface, set by distances colored white. Arrows show the order of updating the values. Bricks without surface (grey) are ignored by accelerated redistancing.

of distance updates across empty space between bricks by additionally using extrapolation between brick faces. For each face of a brick without a direct neighbor, the nearest neighbor bricks are found, and one of their faces is selected for extrapolation. These neighbors are determined once, and after each fast sweeping step, values are extrapolated considering the grid spacing. From the extrapolated and previous values, the one with the smallest absolute value is chosen. To propagate the updated distances, the fast sweeping method is always applied as the final step of this algorithm.

However, using SBS instead of a grid and Newton’s method for intersection enables certain optimizations to the redistancing algorithm for the needs of differentiable rendering. First, it ignores the bricks without surface, and second, the value updates are localized within the bricks and not propagated between them. This accelerated version neither degrades reconstruction quality nor affects convergence to the reference.

The first version outputs correct sparse SDF, so it is used in SparseDR before upsampling and saving the result of reconstruction. The second version is faster while giving the same result for differentiable rendering, so it is applied on each iteration.

Sparsification and Upsampling. The sparsification step is performed before upsampling. During this stage, all bricks containing the surface are preserved, as well as all bricks within a radius of 1 voxel from them, including diagonal neighbors. If any of the adjacent bricks are missing, they are added at this stage. This is necessary in cases where the reconstructed object contains fine details, so that the bricks where these details’ reconstruction should reside would otherwise be absent. Thus, this step not only contributes to reducing the model size, but also eliminates a potential limitation compared to the baseline method. After the sparsification step, the full version of redistancing is applied to recompute all distances before upsampling or saving the resulting SBS, ensuring that the model represents a valid SDF. The upsampling step doubles the resolution of the remaining bricks, generating eight new bricks from each original one. The new distance values are computed using trilinear interpolation.

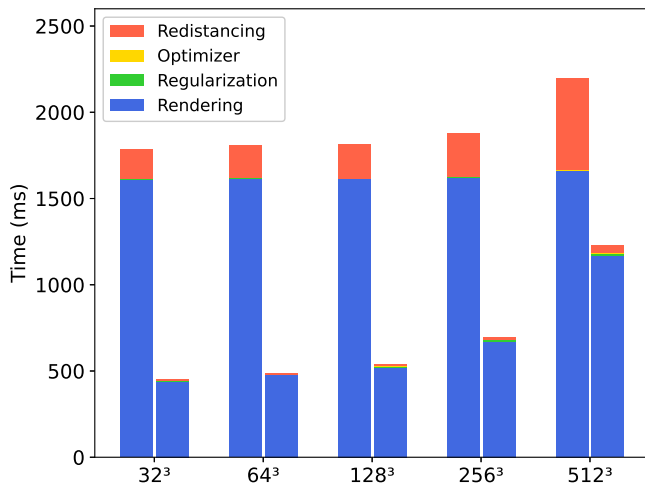


Figure 3: Average time of 1 iteration. For each grid size, the left bar shows the baseline, the right – SparseDR for the SBS of equivalent level of detail.

4 Experiments

Experiments with the baseline method were conducted using the official implementation from [Wang, 2024]. The baseline method employed the hyperparameters specified in the paper [Wang *et al.*, 2024], while the remaining parameters were taken from the configuration file “turbo.json”. The only modifications concerned the number of viewpoints, the batch size, and the upsampling iterations. For consistency across experiments, an identical lighting setup was used: two directional light sources with directions $(1, 1, 1)$ and $(-1, -1, -1)$, along with ambient light. The experiments were carried out on six models. Reconstruction used 16 viewpoints uniformly distributed over a sphere around the scene, generated by the algorithm from [Wang, 2024]. The batch size is 4 viewpoints, image resolution is 1024×1024 . The full optimization process comprised 1000 steps, with upsampling applied at steps 200, 400, 600, 700, 800, and 900. Computations were performed on an AMD Ryzen 9 7950X CPU and an NVIDIA GeForce RTX 4090 GPU.

Figure 3 shows the average reconstruction time across all models for each stage between upsampling steps for SparseDR and the baseline method. The stages are labeled by the SDF grid resolution in voxels corresponding to each level of detail. “Rendering” refers to consecutive rendering of all views within the current batch. Figure 4 presents the mean reconstruction quality across viewpoints for SparseDR and the baseline method. Finally, Figure 1 illustrates the model sizes for SparseDR and the baseline method obtained after each upsampling step. For the baseline method it shows the grid size, and for SparseDR both the size of the SBS and of its BVH. The results demonstrate that the growth of the SBS size is an order of magnitude smaller than that of the SDF grid.

5 Conclusion

In this work we present SparseDR, a system for 3D reconstruction based on the method of differentiable rendering for

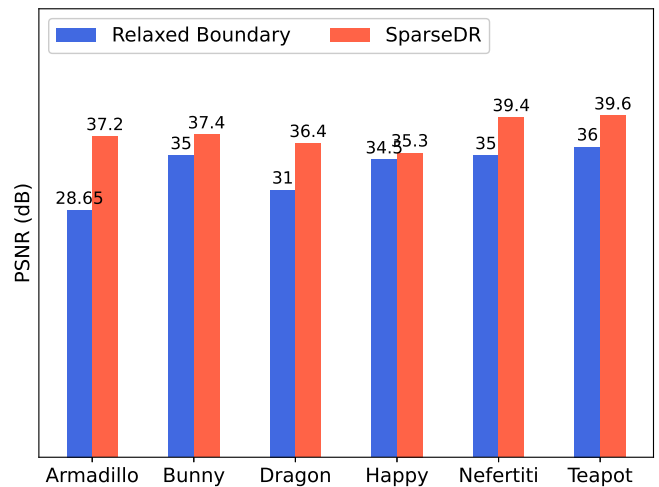


Figure 4: Average PSNR values across viewpoints for models.

Model	32^3	128^3	512^3	1024^3	2048^3
Baseline	0.125	8	512	–	–
Ours(Armadillo)	0.5	6.3	96.4	392.1	1597.7
Baseline	0.125	8	512	–	–
Ours(Bunny)	0.5	6.3	96.4	392.1	1597.7
Baseline	0.125	8	512	–	–
Ours(Dragon)	0.5	6.3	96.4	392.1	1597.7
Baseline	0.125	8	512	–	–
Ours(Happy)	0.5	6.3	96.4	392.1	1597.7
Baseline	0.125	8	512	–	–
Ours(Nefertiti)	0.5	6.3	96.4	392.1	1597.7
Baseline	0.125	8	512	–	–
Ours(Teapot)	0.5	6.3	96.4	392.1	1597.7

Table 1: Model sizes for Relaxed Boundary and SparseDR methods. All values are in MB.

a sparse SDF representation. Using [Wang *et al.*, 2024] as a baseline, we leverage the advantages of SBS and modify all stages of the algorithm. As a result, SparseDR achieves performance and reconstruction quality comparable to the original approach while reducing memory consumption by an order of magnitude. Future work will focus on upgrading SparseDR to a full-fledged path tracer and addressing the challenges that complicate the practical use of differentiable rendering, in particular the need for precise camera parameter information.

References

- [Detrixhe *et al.*, 2013] Miles Detrixhe, Frédéric Gibou, and Chohong Min. A parallel fast sweeping method for the eikonal equation. *Journal of Computational Physics*, 237:46–55, 2013.
- [Jakob *et al.*, 2022] Wenzel Jakob, Sébastien Speierer, Nicolas Roussel, and Delio Vicini. Dr.jit: a just-in-time com-

216 piler for differentiable rendering. 41(4), 2022.

217 [Jason and Yang, 2024] Jason and Edward He et al. Yang.
 218 Pytorch 2: Faster machine learning through dynamic
 219 python bytecode transformation and graph compilation. In
 220 *Proceedings of the 29th ACM International Conference*
 221 *on Architectural Support for Programming Languages*
 222 *and Operating Systems, Volume 2*, ASPLOS '24, page
 223 929–947, New York, NY, USA, 2024. Association for
 224 Computing Machinery.

225 [Söderlund *et al.*, 2022] Herman Hansson Söderlund, Alex
 226 Evans, and Tomas Akenine-Möller. Ray tracing of signed
 227 distance function grids. *Journal of Computer Graphics*
 228 *Techniques Vol*, 11(3), 2022.

229 [Vicini *et al.*, 2022] Delio Vicini, Sébastien Speierer, and
 230 Wenzel Jakob. Differentiable signed distance function ren-
 231 dering. *ACM Trans. Graph.*, 41(4), July 2022.

232 [Wang *et al.*, 2024] Zichen Wang, Xi Deng, Ziyi Zhang,
 233 Wenzel Jakob, and Steve Marschner. A simple approach to
 234 differentiable rendering of sdf. In *SIGGRAPH Asia 2024*
 235 *Conference Papers*, New York, NY, USA, 2024. Associa-
 236 tion for Computing Machinery.

237 [Wang, 2024] Zichen Wang. A simple approach to differen-
 238 tiable rendering of sdf. repository, 2024.

239 [Zhang *et al.*, 2025] Ziyi Zhang, Nicolas Roussel, and Wen-
 240 zel Jakob. Many-worlds inverse rendering. *ACM Trans.*
 241 *Graph.*, 45(1), October 2025.